

Customer Chatbot Architecture & Step-by-Step Build Guide

A source-aligned guide to the public storefront assistant, catalogue discovery, cart, order collection, reviews, voice input and Firestore persistence.

Audience: a learner rebuilding the feature from scratch with React, Flask and Firestore

CURRENT PROJECT IMPLEMENTATION • AUGUST 2026

Workspace: D:\Documents\orderflow\vendly-lk-web

How to use this guide

Build the feature in order. Each section explains the reason first, shows a small code fragment, and then gives a verification step. Paths are relative to the Vendly project root unless an absolute path is shown.

Part	Purpose
1	Architecture and request lifecycle
2	Project files and prerequisites
3	Public API and session security
4	Firestore session and message model
5	Conversation state machine
6	Product discovery, details and alternatives
7	Cart and order collection
8	Order creation and post-order enquiries
9	React storefront, polling and voice
10	Reviews, media, testing and deployment

Learning rule

Type the fragments yourself, run the verification after each stage, and commit only when that stage works. Do not paste the entire feature at once.

1. Architecture and request lifecycle

The Vendly customer chatbot is a hybrid assistant. Deterministic Python rules own all transactional behavior; an optional AI provider only answers product questions and improves natural-language understanding. This separation protects stock, customer information and order creation from hallucinated actions.

End-to-end architecture

```

Customer browser (React StorefrontPage)
  |
  | HTTPS JSON + X-Chat-Session-Token
  v
Flask public blueprint (/api/v1/public)
  |
  +--> public_chat_service.py [state machine + validation]
  |
  |   +--> public_catalog_service.py
  |   +--> ai_service.py [optional product answer]
  |   +--> review_service.py
  |   +--> order_service.py
  |
  v
Firestore
  publicChatSessions/{sessionId}
  publicChatSessions/{sessionId}/messages/{messageId}
  businesses/{businessId}/products, productVariants, orders, customers

```

Layer	Responsibility	Trust boundary
React	Displays messages/cards, keeps a temporary cart, polls for seller replies, captures voice.	Never decides price, stock or final total.
Flask route	Accepts HTTP input, applies rate limits and passes a verified session to services.	Rejects malformed or unauthorized requests.
Chat service	Owns states, parses required details, selects products and triggers an order.	Only accepted state transitions are written.
Domain services	Read products/reviews and create orders through shared business logic.	Server recalculates totals and validates stock.
Firestore	Stores session, messages, products, customer and order records.	Admin SDK is backend-only.

Core invariant

The browser may suggest a cart, but `create_order()` must re-read authoritative variants and compute totals on the server.

2. Project files and prerequisites

Current file	Why it exists
frontend/src/pages/StorefrontPage.jsx	Main storefront, chatbot UI, cart, checkout, reviews, voice and polling.
frontend/src/pages/StorefrontPage.css	Responsive catalogue/chat/card/modal styling.
frontend/src/services/publicService.js	Small frontend wrappers for public API calls.
frontend/src/services/apiClient.js	Base URL, Firebase token, JSON parsing and retry behavior.
backend/app/api/public.py	Public Flask endpoints and rate limits.
backend/app/services/public_chat_service.py	Conversation state machine and secure session handling.
backend/app/services/ai_service.py	Provider-neutral optional AI product answers.
backend/app/services/order_service.py	Stock-safe order creation and totals.
backend/app/services/review_service.py	Approved product and seller reviews.

2.1 Install the two applications

PowerShell

```
# Backend
cd D:\Documents\orderflow\vendly-lk-web\backend
python -m venv .venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt

# Frontend
cd D:\Documents\orderflow\vendly-lk-web\frontend
npm install
```

2.2 Configure environment variables

```
# backend/.env
PORT=5000
FRONTEND_ORIGINS=http://localhost:5173,http://127.0.0.1:5173
FIREBASE_PROJECT_ID=your-project-id
FIREBASE_SERVICE_ACCOUNT_PATH=D:\secrets\firebase-admin.json
AI_PROVIDER=none
AI_API_KEY=
AI_MODEL=
RATE_LIMIT_STORAGE_URI=memory://

# frontend/.env
VITE_FIREBASE_API_KEY=...
VITE_FIREBASE_AUTH_DOMAIN=...
VITE_FIREBASE_PROJECT_ID=...
VITE_FIREBASE_APP_ID=...
VITE_API_BASE_URL=http://127.0.0.1:5000/api/v1
```

Do not commit secrets

Commit .env.example files, but keep .env and the Firebase service-account JSON in .gitignore.

3. Public API and session security

Method	Endpoint	Purpose	Rate
GET	/api/v1/public/stores/{code}	Load seller storefront and catalogue.	Public
POST	/api/v1/public/chat/sessions	Create a token-protected conversation.	Public
POST	/api/v1/public/chat/sessions/{id}/messages	Send one customer message and receive assistant output.	60/min
GET	/api/v1/public/chat/sessions/{id}/messages	Load/poll persisted history.	120/min
POST	/api/v1/public/chat/sessions/{id}/orders	Create an order from the collected cart/details.	6/min
POST	/api/v1/public/chat/sessions/{id}/claim	Attach a guest chat to a signed-in customer.	Authenticated
GET	/api/v1/public/products/{code}/reviews	Load approved public reviews.	Public

backend/app/api/public.py

```
@public_blueprint.post("/chat/sessions/<session_id>/messages")
@limiter.limit("60 per minute", key_func=public_chat_key)
def public_chat_message(session_id):
    result = answer_public_message(
        get_firestore_client(),
        session_id,
        request.headers.get("X-Chat-Session-Token", ""),
        get_json_object(),
    )
    return jsonify({"assistant": result})
```

The route is intentionally thin. Business logic belongs in the service so it can be tested without starting Flask.

3.1 Store only a hash of the chat token

```
def token_hash(token):
    return hashlib.sha256(token.encode("utf-8")).hexdigest()

token = secrets.token_urlsafe(32)
session_reference.set({
    "tokenHash": token_hash(token),
    "status": "active",
    "state": "browsing",
    "cart": [],
})
return {"id": session_reference.id, "sessionToken": token}
```

The raw token is returned once to the browser and kept in sessionStorage/localStorage. Later requests hash the supplied token and compare it with hmac.compare_digest(), reducing exposure if Firestore data is leaked.

4. Firestore session and message model

Path / field	Type	Meaning
publicChatSessions/{id}	document	One conversation owned by a store/product short link.
businessId / productId	string	Tenant scope and optional product-link restriction.
tokenHash	string	SHA-256 hash of the browser session token.
customerUid	string	Firebase UID after guest session is claimed.
state	string	Current deterministic conversation step.
cart	array	Only variantId and positive quantity.
customerDraft	map	Name, phones, address, district, city and note while collecting.
orderId	string	Linked completed order, kept for status questions.
messages/{messageId}	subcollection	Role, text, metadata and createdAt timestamp.

```
def save_chat_message(session_reference, role, message, metadata=None):
    session_reference.collection("messages").document().set({
        "role": role,
        "message": message,
        "metadata": metadata or {},
        "createdAt": firestore.SERVER_TIMESTAMP,
    })
```

Why a subcollection?

Messages can grow independently without making the session document exceed Firestore's document size limit.

5. Conversation state machine

Allowed state transitions

```
browsing
-> collecting-name
-> collecting-phone
-> collecting-secondary-phone
```

```

-> collecting-address
-> collecting-district
-> collecting-nearest-city
-> collecting-delivery-note
-> awaiting-confirmation
-> completed

```

```

completed -- explicit "another order" --> browsing
completed -- status question -----> completed (same order context)

```

State	Accepted input	Next state
browsing	Product/category questions, Add buttons, or explicit order intent.	Browsing or collecting-name
collecting-name	A plausible customer name.	collecting-phone
collecting-phone	Valid Sri Lankan primary number.	collecting-secondary-phone
collecting-secondary-phone	Valid second number or skip phrase.	collecting-address
collecting-address	Street/delivery address.	collecting-district
collecting-district	District.	collecting-nearest-city
collecting-nearest-city	Nearest city/town.	collecting-delivery-note
collecting-delivery-note	Note or none.	awaiting-confirmation
awaiting-confirmation	Confirm or correction request.	completed or earlier state
completed	Order status/info or explicit new order.	completed or browsing

A single response helper keeps persistence consistent

```

def respond(text, action, *, next_state=None, metadata=None):
    state = next_state or current_state
    save_chat_message(session_reference, "assistant", text, {
        "action": action,
        "state": state,
        **(metadata or {}),
    })
    session_reference.update({
        "state": state,
        "cart": cart,
        "customerDraft": customer_draft,
        "updatedAt": firestore.SERVER_TIMESTAMP,
    })
    return {"message": text, "action": action, "state": state,
            "cartSummary": cart_summary}

```

6. Product discovery, details and alternatives

1. Load only active products belonging to the session business.
2. Try exact/product-token matching with `find_product_in_message()`.
3. If the message is a category request, return all products in that category.
4. For one specific product, return its photos, description and approved reviews.
5. Offer related products from the same category only after the customer is not satisfied.
6. Do not add anything to cart until the customer clicks Add or clearly asks to order it.

```

def related_products(products, selected_product, limit=4):
    return [
        product for product in products
        if product["id"] != selected_product["id"]
        and product.get("categoryId") == selected_product.get("categoryId")
   ][:limit]

```

6.1 Optional AI answers must be fact-bounded

```

def product_prompt(question, product):
    facts = {
        "name": product.get("name"),
        "brand": product.get("brand"),
        "description": product.get("description"),
        "sellerAiDescription": product.get("aiDescription"),
        "priceLkr": product.get("sellingPriceMinor", 0) / 100,
        "availableSizes": [v.get("size") for v in product.get("variants", [])],
    }

```

```
"approvedReviewSnippets": product.get("approvedReviewSnippets", []),
}
return f"Use only PRODUCT FACTS. Keep the answer brief.\nQUESTION: {question}\nPRODUCT FACTS: {facts}"
```

No AI web browsing in the order transaction

If a specification is missing, say it is not confirmed or use a separately governed product-enrichment workflow. Never invent waterproofing, SIM, warranty or health claims.

7. Cart and order collection

7.1 Normalize browser cart input

```
def normalize_chat_cart(value):
    result = []
    for item in value if isinstance(value, list) else []:
        variant_id = str(item.get("variantId") or "").strip()
        try:
            quantity = int(item.get("quantity") or 0)
        except (TypeError, ValueError):
            continue
        if variant_id and 0 < quantity <= 1000:
            result.append({"variantId": variant_id, "quantity": quantity})
    return result
```

The server then looks up every variant and removes unknown IDs. It returns cartSummary so the UI displays server-approved names, images, prices, quantities and line totals.

7.2 Begin detail collection only after selection is finished

```
if current_state == "browsing" and cart_summary and is_finished_selecting_items(message):
    return respond(
        f"Great. Your draft contains {sum(i['quantity'] for i in cart_summary)} item(s). What is your name?",
        "collect-name",
        next_state="collecting-name",
    )
```

7.3 Validate customer details one or two at a time

```
if current_state == "collecting-phone":
    try:
        phone = normalize_sri_lankan_phone(message)
    except ValueError as error:
        return respond(str(error), "collect-phone", next_state="collecting-phone")
    customer_draft["phoneNumber"] = phone
    return respond(
        "Do you have a second phone number? Send it, or type skip.",
        "collect-secondary-phone",
        next_state="collecting-secondary-phone",
    )
```

User experience rule

Ask for missing information progressively. The required order data is product, quantity, name, primary phone, address, district and nearest city; the second phone and note are optional.

8. Order creation and post-order enquiries

8.1 Confirm before writing

```
if current_state == "awaiting-confirmation":
```

```

if normalized_phrase(message) in CONFIRMATION_PHRASES:
    order = create_public_chat_order(database, session_id, provided_token, {
        "items": cart,
        "customer": customer_draft,
        "deliveryNote": customer_draft.get("deliveryNote", ""),
    })
    # Persist order link, clear cart, retain customer summary.
    return respond(
        f"Your order {order['orderNumber']} was placed successfully.",
        "order-confirmed",
        next_state="completed",
    )

```

`create_public_chat_order()` finds or creates the customer, builds an order payload, enforces product-link restrictions, calls the shared `create_order()` service, and saves `orderId` on the chat session.

8.2 Keep a completed chat open for support

```

if current_state == "completed" or session.get("status") == "completed":
    if is_explicit_new_order_request(message):
        # Clear only the cart and start a new order flow.
        return respond("What product would you like to order?", "prompt-product",
            next_state="browsing")
    if is_order_enquiry(message):
        order = latest_order_for_session(database, session)
        return respond(order_information_message(order), "show-order-info",
            next_state="completed")
    return respond("Your order is saved. Ask me for its status or say 'another order'.",
        "order-help", next_state="completed")

```

Bug prevention

Do not mark the conversation closed after checkout. Keep `status=completed` accepted by authorization; only an explicit closed/disabled state should block messaging.

9. React storefront, polling and voice

9.1 Keep API wrappers small

`frontend/src/services/publicService.js`

```

export function sendPublicChatMessage(sessionId, sessionToken, data) {
    return apiRequest(`/public/chat/sessions/${sessionId}/messages`, {
        method: "POST",
        body: data,
        headers: { "X-Chat-Session-Token": sessionToken },
        requiresAuthentication: false,
    });
}

```

9.2 Send cart with every chat request

```

const response = await sendPublicChatMessage(session.id, session.sessionToken, {
    message: cleanMessage,
    cart: cart.map((item) => ({
        variantId: item.variantId,
        quantity: item.quantity,
    })),
});

setSession((current) => ({ ...current, state: response.assistant.state }));
setMessages((current) => [...current, {
    role: "assistant",
    text: response.assistant.message,
    action: response.assistant.action,
    products: response.assistant.products,
}]]);

```

9.3 Poll for seller and status messages

```
useEffect(() => {
  if (!session) return undefined;
  const timer = window.setInterval(async () => {
    const result = await getPublicChatMessages(session.id, session.sessionToken);
    appendOnlyUnseenMessages(result.messages);
  }, 4000);
  return () => window.clearInterval(timer);
}, [session]);
```

The current implementation tracks received message IDs in a ref, preventing duplicates. For larger scale, replace polling with a controlled realtime channel or Firestore listener exposed through secure rules.

9.4 Add voice input as progressive enhancement

```
const SpeechRecognition = window.SpeechRecognition || window.webkitSpeechRecognition;
if (!SpeechRecognition) {
  setErrorMessage("Voice input is not supported in this browser.");
  return;
}
const recognition = new SpeechRecognition();
recognition.lang = voiceLanguage; // en-LK or si-LK
recognition.interimResults = false;
recognition.onresult = (event) => {
  const transcript = event.results?.[0]?.[0]?.transcript?.trim();
  if (transcript) sendChatText(transcript);
};
recognition.start();
```

Browser requirement

Camera/microphone features normally require HTTPS or localhost. A phone opening a plain LAN IP may block them even when the code is correct.

10. Reviews, media, testing and deployment

10.1 Return only approved reviews

```
reviews = list_public_product_reviews(database, product["id"])
seller_rating = list_publicSellerReviews(database, session["businessId"])
return respond(
  "Here are verified reviews for this product and the seller rating.",
  "show-reviews",
  metadata={"reviews": reviews, "sellerRating": seller_rating},
)
```

Review images should be stored as Cloudinary HTTPS URLs. Render them as links with target="_blank" and rel="noreferrer" so the full image opens safely in a new tab.

10.2 Minimum automated tests

Test	Expected result
Create session with valid store code	Returns sessionId + raw token; Firestore stores only tokenHash.
Wrong session token	401/403; no message or order is written.
Unknown variant in cart	Removed or rejected; never affects stock.
Invalid phone/address	State remains at the same collection step.
Multiple products	One order with many items, not separate orders.
Confirm order	Order created once; stock reserved once; totals server-computed.
Ask status after checkout	Returns linked order status; does not reset catalogue.
Say another order	Starts a clean browsing flow while keeping history.
AI provider unavailable	Deterministic catalogue/order flow still works.

10.3 Run and verify

```
# Terminal 1
cd backend
.\.venv\Scripts\Activate.ps1
python run.py

# Terminal 2
cd frontend
npm run dev -- --host 0.0.0.0

# Backend tests
cd backend
pytest -q

# Production frontend check
cd frontend
npm run build
```

10.4 Troubleshooting checklist

Symptom	Check
Failed to fetch	Backend running, VITE_API_BASE_URL, CORS origin, Windows firewall and same LAN.
Chat session is closed	Authorization accepts active and completed; checkout did not set closed.
Cards appear but cart is empty	Add handler updates cart and sends variantId/quantity on next message.
Images are cropped	Use object-fit: contain; fixed media area; preserve source aspect ratio.
Messages require refresh	Polling interval active and unseen IDs are merged, not overwritten.
AI replies incorrectly	Keep transaction states deterministic and reduce prompt scope to product facts.
Quota errors	Use Firestore emulator locally and avoid polling too frequently.

Completion definition

The chatbot is complete only when it can discover products, answer fact-bounded questions, collect a multi-item order, confirm once, persist history, show later status updates, and continue support after checkout.